Critique DPO

Zihe Liu, Peter Hayes

February 28, 2026

Abstract

This is *very much* a work-in-progress document, which reviews how generative verifiers can be implemented and simple experimental results. Much of this current version are disjointed concepts and messy; my current task is to tidy these reviews up and include the experimental results.

1 Introduction

As large language models (LLMs) become more capable, fine-tuning techniques to align with human intent are increasingly important. A key consideration for aligning these models is how to most effectively use human resources, or model resources in the case where LLMs themselves are used as oracles. Reinforcement learning from Human or AI preferences (RLHF/RLAIF) is the most prominent example of such a technique but is complex and often unstable. Direct Preference Optimization (DPO) has recently been proposed as a simpler and more stable alternative. In this note, we explore how to expand the DPO objective to make better use of feedback data from the human or AI oracles. Specifically, we incorporate the Chain-Of-Thought reasoning of a generative verifier (acting to verify and critique the validity of base models) in the DPO loss term for the base model.

Asking these oracles to provide reasoning for their binary preferences is common practice. In particular, for LLM-as-a-judge style workflows asking the model to provide reasoning (or so-called chain-of-thought) has been shown to actually improve the model predictions, or verifications in the case of generative verifiers. How can we leverage this rationale when fine-tuning our target model to improve performance in a more data-efficient manner?

2 Background: DPO

For a given prompt x, two completions are sampled from the policy $(y_w, y_l) \sim \pi_\theta(y|x)$ and an oracle judges which it prefers. We denote y_w as the preferred completion and y_l as the other. This process is repeated over N prompts resulting in the pairwise preference dataset $\mathcal{D} = \{x^{(i)}, y_w^{(i)}, y_l^{(i)}\}_{i=1}^N$.

2.1 Bradley-Terry Preference Model

We can model the judge preferences using the Bradley-Terry (BT) model, which expresses the probability that completion y_1 is preferred over y_2 under a latent reward function r^*:

$$p^*(y_1 \succ y_2 \mid x) = \frac{\exp\left(r^*(x, y_1)\right)}{\exp\left(r^*(x, y_1)\right) + \exp\left(r^*(x, y_2)\right)} = \sigma(r^*(x, y_1) - r^*(x, y_2))$$

A key property is that BT depends only on the *difference* of rewards, not their absolute values.

2.2 Standard RLHF

The standard RLHF pipeline comprises 2 stages. First, we learn a reward model r_ϕ where we find the maximum likelihood estimate (MLE) of ϕ under our preference dataset $\mathcal{D}$ and BT model - termed reward modelling. Then, we use RL (PPO) to learn an optimal policy π_θ for this learned reward model.

2.2.1 Reward Modelling

Since r^* is unknown, we learn a parametric approximation r_ϕ. Under the BT model, the likelihood of the observed preference dataset is $\ell(\mathcal{D} \mid \phi) = \prod_{i=1}^{N} \sigma\Big(r_\phi(x^{(i)}, y_w^{(i)}) - r_\phi(x^{(i)}, y_l^{(i)})\Big)$. Taking the log and writing as an expectation, the MLE is obtained by minimising

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{\mathcal{D}}\left[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))\right] \tag{1}$$

2.2.2 RL Fine-Tuning

The learned reward model is then used to fine-tune the policy. The aim is to maximise the KL-constrained objective

$$\max_{\pi_\theta} \; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(y|x)}\left[r_\phi(x, y)\right] - \beta\, D_{\text{KL}}(\pi_\theta(y|x) \| \pi_{\text{ref}}(y|x)) \tag{2}$$

where π_{ref} is the reference policy (typically the SFT model), kept frozen. The KL term prevents the policy from deviating too far from the distribution on which the reward model is accurate, and $\beta > 0$ controls this trade-off. In practice this is optimised with PPO, requiring four models in memory: the policy, reference, reward model, and value function.

2.3 DPO

DPO [Rafailov et al., 2024] bypasses the reward modelling and RL steps entirely through a change of variables. The key insight is that there exists a bijection between reward functions and their corresponding optimal policies under the KL-constrained objective (2), and this bijection can be exploited to reparameterise the preference likelihood directly in terms of the policy. We follow the expositions by [Wolfe, 2025] and [Romero, 2024] in our presentation of the derivation below.

2.3.1 Optimal Policy

For *any* reward function r (not just the learned r_ϕ or the latent r^*), the KL-constrained objective (2) can be rewritten as a minimisation of a KL divergence between π and a target distribution defined by r, plus a constant. Specifically, letting $Z_r(x) = \sum_y \pi_{\text{ref}}(y|x) \exp\Big(\frac{1}{\beta} r(x, y)\Big)$ be a partition function that depends on x and r but is constant with respect to π, the objective becomes

$$\min_{\pi} \; \mathbb{E}_x \left[D_{\text{KL}}\left(\pi(y|x) \;\Big\|\; \frac{1}{Z_r(x)}\, \pi_{\text{ref}}(y|x)\, \exp\Big(\tfrac{1}{\beta}\, r(x, y)\Big)\right) - \log Z_r(x) \right]$$

By Gibbs' inequality ($D_{\text{KL}} \geq 0$, with equality iff the distributions are identical), the unique optimum is

$$\pi_r(y|x) = \frac{1}{Z_r(x)}\, \pi_{\text{ref}}(y|x)\, \exp\left(\frac{1}{\beta}\, r(x, y)\right) \tag{3}$$

This defines a bijection: every reward r maps to a unique optimal policy π_r, and conversely every policy π_r defines a unique reward[1] r for which it is optimal. Using the latent reward r^* gives the optimal policy $\pi^* \equiv \pi_{r^*}$ for our latent rewards.

2.3.2 Reparameterisation

Rearranging (3) for the reward expresses any reward purely in terms of its corresponding optimal policy and a prompt-dependent partition function:

$$r(x, y) = \beta \log \frac{\pi_r(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z_r(x)$$

[1] We really mean unique equivalence class of rewards and its optimal policy; see Lemma 1 and Lemma 2 in [Rafailov et al., 2024].

Substituting the latent reward $r^*(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z_{r^*}(x)$ into the BT model, the $\beta \log Z_{r^*}(x)$ terms cancel because BT depends only on reward *differences* and $Z_{r^*}(x)$ is independent of y:

$$p^*(y_w \succ y_l \mid x) = \sigma\left(\beta \log \frac{\pi^*(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi^*(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)$$

The preference model now has no dependence on a reward model — it is expressed entirely in terms of policies.

2.3.3 DPO Loss

We replace the fixed π^* with a learnable π_θ and maximise the likelihood of the observed preferences, conditioned on our policy parameters θ. Defining the *implicit reward*

$$\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} \tag{4}$$

the DPO loss is

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{\mathcal{D}}\left[\log \sigma\left(\hat{r}_\theta(x, y_w) - \hat{r}_\theta(x, y_l)\right)\right] \tag{5}$$

This is identical in form to the reward model loss (1), but conditioned on the policy parameters θ instead of the reward model parameters ϕ. Traditionally, RLHF performs MLE on $\mathcal{D}$ conditioned on ϕ to learn a reward model, then uses RL to find the optimal policy for that reward. DPO instead performs MLE on $\mathcal{D}$ conditioned directly on θ. Because of the reward–policy bijection, improving π_θ to better fit the preference data simultaneously yields an implicit reward $\hat{r}_\theta$ for which π_θ is guaranteed to be optimal. This eliminates both the reward modelling step and the RL step, requiring only two models in memory (the policy π_θ and the frozen reference π_{ref}).

In this work we suggest simple extensions to this objective.

3 Literature Review

We review research incorporating reasoning and critique data into fine-tuning, building toward the critique-augmented preference optimisation approach of this work. We adopt a unified notation: π_θ denotes the learnable policy, π_{ref} the frozen reference (usually the base SFT model), x the prompt, y_w and y_l the preferred and rejected responses, c a model's own chain-of-thought reasoning trace, c_v an oracle or verifier chain-of-thought critique ("CoT of the verifier"), $r_\phi(x, y)$ a learned scalar reward, and $\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ the implicit reward.

The review is organised around a central question: *what kind of signal is being used, and for which model?* The narrative moves from binary AI feedback (Section 3.1), through enriching DPO with supervised reasoning targets (Section 3.2), to methods that improve verifiers by having them reason (Section 3.3), and finally to the closest work — methods that bring verifier critique directly into the base model's training objective (Section 3.4).

3.1 Beyond Binary Feedback

3.1.1 RLAIF and the Information Bottleneck

RL from AI Feedback (RLAIF) replaces human annotators with LLM oracles. [Bai et al., 2022] first use LLM-generated labels for harmlessness finetuning, doing RL with a preference model trained with LLM preferences. [Lee et al., 2024] use LLM to directly give rewards, showing that RLAIF matches or exceeds RLHF on summarisation and dialogue, even when the labeller is the same model being aligned. [Li et al., 2024] instruct the labeller to reason explicitly and report modest improvements. A shared limitation is that the oracle's reasoning is discarded after labelling: the sole training signal is a binary or scalar preference label, creating an information bottleneck that the remainder of this review seeks to address.

3.1.2 Impact of Chain-of-Thought Data for DPO

[Liu et al., 2025] provide a systematic empirical study of CoT's impact on DPO. In text-to-SQL — where preference datasets traditionally contain only final SQL queries *without* reasoning steps — DPO degrades performance across the majority of ten tested LLMs. Augmenting the same datasets with synthetically generated chain-of-thought solutions achieves consistent DPO improvements for the first time. The analysis identifies three mechanisms. First, *reward hacking mitigation*: without CoT, the implicit reward (4) can be inflated by superficial correlates (longer SQL, non-executable patterns) that do not reflect reasoning quality. Model performance initially peaks then decreases, while the implicit reward continues to rise. With CoT, the model's self-assessed implicit reward tracks actual performance. Second, *discriminative calibration*: CoT-trained models maintain stable classification accuracy between correct and incorrect solutions throughout training, whereas vanilla DPO becomes unstable. Third, *sample efficiency*: CoT-augmented DPO continues to improve with additional data, while vanilla DPO saturates. 2 samples of CoT with DPO achieves the same performance gains as 16 samples of vanilla DPO.

This finding directly motivates our approach: if the model's own CoT improves DPO, then an external verifier's critique — which provides a richer perspective on why one response is preferred — should be informative too.

3.1.3 Self-Taught Reasoning: STaR and V-STaR

STaR. [Zelikman et al., 2022] establish that CoT data, even when self-generated, is a richer training signal than outcome labels alone. Given a pretrained model M and a dataset $\mathcal{D} = \{(x_i, y_i)\}$ of problems with ground-truth answers, STaR begins with a small few-shot prompt set $\mathcal{P}$ containing rationale examples. At each iteration, the model generates a reasoning trace $\hat{c}_i$ and answer $\hat{y}_i$ for every problem, using $\mathcal{P}$ in the context. Only correct generations where $\hat{y}_i = y_i$ are retained; the model is then fine-tuned on this filtered set with the standard NLL objective on rationale-answer pairs:

$$\mathcal{L}_{\text{STaR}}(M) = -\mathbb{E}_{(x,y)\sim\mathcal{D}}\left[\mathbf{1}[\hat{y} = y] \cdot \sum_t \log M(\hat{c}_t, \hat{y}_t \mid x, \hat{c}_{<t})\right] \tag{6}$$

STaR with rationalization matches GPT-3 (30× larger) on CommonsenseQA while using only 86.7% of training data. The method demonstrates that self-generated rationales – even produced with $|\mathcal{P}| = 10$ few-shot examples – provide a dramatically richer training signal than training on final answers alone.

V-STaR. [Hosseini et al., 2024] identify the core inefficiency of STaR: the large number of *incorrect* self-generated solutions is discarded, despite containing information about error patterns. V-STaR repurposes these failures to co-train a generator G and a verifier V in parallel across T iterations.

They maintain two accumulating datasets: $\mathcal{D}_{\text{GEN}}$: correct solutions $(x, \hat{y})$, used for generator SFT, and $\mathcal{D}_{\text{VER}}$: *all* generated solutions $(x, \hat{y}, z)$, used for verifier training. At each iteration t, $k = 16$ solutions per problem are sampled from G^t and automatically labeled for correctness — by comparing the extracted final answer against the ground-truth answer for math, or by running test cases for code. Correct solutions are appended to $\mathcal{D}_{\text{GEN}}$; all solutions are appended to $\mathcal{D}_{\text{VER}}$. G^t is fine-tuned on $\mathcal{D}_{\text{GEN}}^{<t}$ with the standard NLL loss Eq. (6). From $\mathcal{D}_{\text{VER}}$, preference pairs (x, y^+, y^-) are formed by taking the pair of all correct solutions y^+ and all incorrect solutions y^- for each problem x. The verifier is trained with DPO using G_{SFT} (the base model fine-tuned once) as the fixed reference policy:

$$\mathcal{L}_{\text{DPO}}(V^t; G_{\text{SFT}}) = -\mathbb{E}_{(x,y^+,y^-)\sim\mathcal{D}_{\text{VER}}}\left[\log\sigma\big(\hat{r}(x, y^+) - \hat{r}(x, y^-)\big)\right], \quad \hat{r}(x, y) = \beta \log \frac{V^t(y \mid x)}{G_{\text{SFT}}(y \mid x)} \tag{7}$$

At inference time, the verifier is used in Best-of-k reranking to choose solutions for better performance. Crucially, the verifier provides no training-time signal to the generator. The generator is trained purely via SFT on correct solutions (identical to STaR). The verifier's direct contribution is better performance at inference via Best-of-k reranking. As the generator improves across iterations it produces harder-to-distinguish incorrect solutions, enriching $\mathcal{D}_{\text{VER}}$. However this does not feed any gradient back into the generator's training. This distinguishes V-STaR from later methods. Nevertheless, V-STaR 7B surpasses LLaMA-2 70B (8-shot) on GSM8K and nearly matches CodeLLaMA 34B on HumanEval in out-of-domain transfer, which illustrates the value of CoT training signals.

3.2 DPO with NLL

A recurring structural pattern in the literature combines the DPO preference loss with an auxiliary negative log-likelihood (NLL) term on a supervised target. We present the generic template first, then show how each paper instantiates it with a **different NLL target** t:

$$\mathcal{L}_{\text{DPO+NLL}} = \underbrace{-\log \sigma(\hat{r}_\theta(x, y_w) - \hat{r}_\theta(x, y_l))}_{\text{preference term}} + \alpha \underbrace{\mathcal{L}_{\text{NLL}}(t \mid x)}_{\text{supervised anchor}} \tag{8}$$

The DPO term provides a contrastive signal (push apart y_w and y_l), while the NLL term anchors the policy to a specific supervised target. As we show below, the choice of t is what distinguishes these methods from one another.

3.2.1 Iterative Reasoning Preference Optimization (IRPO)

[Pang et al., 2024] set the NLL target to the *model's own winning chain-of-thought*: $t = (c_w, y_w)$. At each iteration, the model samples N reasoning traces per prompt, partitions them into correct (c_w, y_w) and incorrect (c_l, y_l) by checking the final answer against ground truth, and optimises:

$$\begin{aligned}\mathcal{L}_{\text{IRPO}} &= \mathcal{L}_{\text{DPO}}(c_w, y_w, c_l, y_l \mid x) + \alpha\, \mathcal{L}_{\text{NLL}}(c_w, y_w \mid x) \\ &= -\log \sigma\left(\beta \log \frac{\pi_\theta(c_w, y_w \mid x)}{\pi_{\text{ref}}(c_w, y_w \mid x)} - \beta \log \frac{\pi_\theta(c_l, y_l \mid x)}{\pi_{\text{ref}}(c_l, y_l \mid x)}\right) - \alpha \frac{\log \pi_\theta(c_w, y_w \mid x)}{|c_w| + |y_w|}\end{aligned} \tag{9}$$

The NLL term is length-normalised by $|c_w| + |y_w|$ to avoid a bias toward shorter sequences. Crucially, the reference model π_{ref} is updated to π_θ at each iteration before re-sampling. Under standard DPO (i.e. $\alpha = 0$), the log-probabilities of *chosen* sequences actually decrease during training — the model achieves the required preference margin by pushing rejected probabilities down faster than chosen ones fall. The NLL anchor prevents this collapse: with $\alpha > 0$, chosen log-probabilities increase while rejected ones decrease, creating a healthy expanding margin. Empirically, DPO alone achieves 61.8% on GSM8K while DPO+NLL achieves 73.1% at iteration 1; after four iterations.

3.2.2 AIPO: Agreement-Aware Iterative Preference Optimization

[Shen et al., 2024] set the NLL target to the **winning response**: $t = y_w$. Their key modification is to the DPO term itself. Defining the log-probability ratios

$$s_\theta = \log \frac{\pi_\theta(y_w \mid x)}{\pi_\theta(y_l \mid x)}, \qquad s_{\text{ref}} = \log \frac{\pi_{\text{ref}}(y_w \mid x)}{\pi_{\text{ref}}(y_l \mid x)},$$

standard DPO is $-\log \sigma(\beta(s_\theta - s_{\text{ref}}))$. AIPO introduces an agreement-aware coefficient $\alpha > 0$:

$$\mathcal{L}_{\alpha\text{-DPO}} = -\mathbb{E}_{\mathcal{D}}\left[\log \sigma\left(\beta\left(s_\theta - (1+\alpha)\, s_{\text{ref}}\right)\right)\right] \tag{10}$$

The gradient weight decomposes as $\sigma(\beta(s_{\text{ref}} - s_\theta + \alpha\, s_{\text{ref}}))$: the additional $\alpha\, s_{\text{ref}}$ amplifies the learning signal when the reference model agrees with the preference label and dampens it when there is disagreement, mitigating the length exploitation that compounds across iterative DPO. The full AIPO loss combines this with a length-normalised NLL term:

$$\mathcal{L}_{\text{AIPO}} = \mathcal{L}_{\alpha\text{-DPO}} + \lambda \cdot \mathcal{L}_{\text{NLL}}, \qquad \mathcal{L}_{\text{NLL}} = -\frac{1}{|y_w|} \log \pi_\theta(y_w \mid x) \tag{11}$$

AIPO achieves 57.1% length-controlled win rate on AlpacaEval 2.0 with Gemma-2-9B-It and 81.6% on Arena-Hard with Mistral-Large, outperforming GPT-4.

3.2.3 Direct Judgement Preference Optimization (DJPO)

[P. Wang et al., 2024] set the NLL target to the **verifier's CoT critique and judgment**: $t = (c_v, j)$, where c_v is the verifier's chain-of-thought critique and j is the final judgment. Crucially, the model being trained is a *judge*, not the base policy. They construct three types of DPO preference pairs — CoT critique $\mathcal{D}_{\text{CoT}}$,

standard judgment $\mathcal{D}_{\text{Std}}$, and response deduction $\mathcal{D}_{\text{Ded}}$ — and train the collective preference dataset with:

$$\mathcal{L}_{\text{DJPO}} = -\frac{\log \pi_\theta(y_w \mid x)}{|y_w| + |x|} - \log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right) \tag{12}$$

where y_w and y_l are correct and incorrect judge evaluations respectively. Using 680K preference pairs with a 70:15:15 ratio across the three types, they train SFR-Judge models at 8B, 12B, and 70B scales. SFR-Judge-8B outperforms GPT-4o on aggregate pairwise evaluation across 7 benchmarks; SFR-Judge-70B achieves the best overall score of 84.25%.

3.3 Generative Verifiers and Critique-Augmented Reward Models

The methods above train the policy model. A parallel line of work improves the verifiers and reward models that judge responses, by having them produce reasoning before scoring. These methods establish that critique data is a powerful training signal for judging. Generative verifiers aim to provide a scalar verifier score that is used during inference for Best-of-N sampling, whereas critique-augmented reward models produce better reward models for downstream preference tuning etc.

3.3.1 Generative Verifiers (GenRM)

[Zhang et al., 2024] observe that discriminative reward models require a separate classification head to assign scalar rewards and cannot use the LLM's generative capabilities — they cannot reason step-by-step before scoring, and they cannot leverage additional inference-time compute. Generative verifiers (GenRM) address this by recasting verification as next-token prediction, training the verifier with the standard SFT loss. Gemini 1.0 Pro generates solutions for GSM8K while a Gemma model is fine-tuned as the verifier.

In the simplest variant (**GenRM-Direct**), given instruction I = "Is the answer correct (Yes/No)?", the verifier is trained on:

$$\mathcal{D}_{\text{Direct}} = \{(x, y^+, I),\ \text{'Yes'}\} \cup \{(x, y^-, I),\ \text{'No'}\} \tag{13}$$

where y^+ and y^- are correct and incorrect solutions labelled by an external oracle. At inference, the verifier score is the next-token probability of 'Yes':

$$r_{\text{Direct}}(x, y) = \pi_\theta(\text{'Yes'} \mid x, y, I) \tag{14}$$

which is used with Best-of-N sampling to choose the best generator solution at inference time.

Unifying generation and verification. Training on correct solution generation provides positive transfer to verification. The default training objective for the verifier mixes both tasks:

$$\mathcal{L}_{\text{GenRM}}(\theta) = \mathcal{L}_{\text{SFT}}(\theta, \mathcal{D}_{\text{verify}}) + \lambda\, \mathcal{L}_{\text{SFT}}(\theta, \mathcal{D}_{\text{correct}}) \tag{15}$$

where $\mathcal{D}_{\text{verify}}$ is either $\mathcal{D}_{\text{Direct}}$ or $\mathcal{D}_{\text{CoT}}$, and $\mathcal{D}_{\text{correct}}$ is a set of correct solutions from the training set.

GenRM-CoT. Since verification requires nuanced reasoning, GenRM-CoT inserts a chain-of-thought rationale c_v before the final Yes/No token. The dataset is:

$$\mathcal{D}_{\text{CoT}} = \big\{(x, y^+, I_v),\ (c_v, I, \text{'Yes'})\big\} \cup \big\{(x, y^-, I_v),\ (c_v, I, \text{'No'})\big\}$$

where I_v = "Let's verify step by step". The rationales c_v are generated offline by a capable LLM. Rationales are filtered by majority agreement across multiple generated samples. The verifier itself generates no c_v during training; only at inference does it sample its own rationale and then score:

$$r_{\text{CoT}}(x, y) = \pi_\theta(\text{'Yes'} \mid x, y, I_v, c_v, I), \qquad c_v \sim \pi_\theta(\cdot \mid x, y, I_v) \tag{16}$$

Multiple independent rationales can be sampled and aggregated via majority voting to use additional test-time compute:

$$r_{\text{MV@}K}(x, y) = \frac{1}{K} \sum_{i=1}^{K} \pi_\theta\Big(\text{'Yes'} \mid x, y, I_v, c_v^{(i)}, I\Big), \quad c_v^{(i)} \overset{\text{i.i.d.}}{\sim} \pi_\theta(\cdot \mid x, y, I_v) \tag{17}$$

The whole procedure is a single-pass offline SFT; the trained verifier is used only at inference for Best-of-N reranking. GenRM-CoT achieves 94.2% Best-of-32 on GSM8K (vs. ∼90% for a discriminative RM), and generalises to MATH500 trained only on GSM8K with 6.4× better sample efficiency than the discriminative baseline. GenRM also outperforms DPO-trained verifiers substantially, with the best DPO scoring strategy (raw log-likelihood, without subtracting the reference) still underperforming GenRM-Direct.

3.3.2 GenRM-STaR-DPO: Training Judges with DPO on Rationales

[Mahan et al., 2024] (Mahan et al., 2024) propose a unified RLHF–RLAIF framework built on a key theoretical observation: under the Bradley-Terry model, the standard RLHF reward-difference objective is equivalent to a preference log-odds objective,

$$r_\phi(x, y) - r_\phi(x, y_{\text{ref}}) = \log \frac{p_{\text{BT}}(y \succ y_{\text{ref}} \mid x)}{1 - p_{\text{BT}}(y \succ y_{\text{ref}} \mid x)} \tag{18}$$

This means one can replace the scalar reward parameterisation with a fully general preference model $p_\phi(y_w \succ y_l \mid x)$ extracted directly from an LLM's token probabilities, without requiring a separate classification head or a specific distributional assumption. This is the conceptual foundation for training a generative judge as a reward model.

Three training variants. Given a preference dataset $\mathcal{D} = \{(x, y_1, y_2, I)\}$ where $I \in \{A, B\}$ is the winning response indicator:

GenRM (no CoT) — standard SFT on the indicator token:

$$\mathcal{L}_{\text{GenRM}}(\pi_\phi) = \mathbb{E}_{(x,y_1,y_2,I)\sim\mathcal{D}}[-\log \pi_\phi(I \mid x, y_1, y_2)] \tag{19}$$

CoT-GenRM with Rationalization — a post-rationalization model q (e.g. GPT-4 or a stronger LLM) generates a critique rationale c_v conditioned on the correct answer, and the model is trained on both:

$$\mathcal{L}_{\text{Rationalizer}}(\pi_\phi) = \mathbb{E}_{(x,y_1,y_2,c_v,I)\sim\mathcal{D}}[-\log \pi_\phi(I \mid x, y_1, y_2, c_v) - \log \pi_\phi(c_v \mid x, y_1, y_2)] \tag{20}$$

CoT-GenRM-STaR-DPO — instead of discarding incorrect self-generated critique rationales (as STaR-SFT does), STaR-DPO constructs preference pairs $(c_{v,w}, I_w)$ vs $(c_{v,l}, I_l)$ from correct and incorrect reasoning traces and applies DPO over the full rationale-plus-judgment sequence:

$$\mathcal{L}_{\text{STaR-DPO}}(\pi_\phi) = -\mathbb{E}_{\mathcal{D}}\left[\log \sigma\left(\beta \log \frac{\pi_\phi(I_w, c_{v,w} \mid x, y_1, y_2)}{\pi_{\text{ref}}(I_w, c_{v,w} \mid x, y_1, y_2)} - \beta \log \frac{\pi_\phi(I_l, c_{v,l} \mid x, y_1, y_2)}{\pi_{\text{ref}}(I_l, c_{v,l} \mid x, y_1, y_2)}\right)\right] \tag{21}$$

The model learns not just which response is correct but which *reasoning patterns* lead to correct judgments, and this contrastive signal is what drives OOD generalisation.

On-policy rationales are essential: using rationales from a strong off-policy model (GPT-4 or Llama-3.1-70B) as the training signal — the STaR-Rationalizer approach — matches STaR-DPO in-distribution but collapses OOD (RewardBench accuracy drops to 63–70%, below even the zero-shot LLM-as-a-Judge). The authors attribute this to distribution mismatch: the model learns to imitate the rationalizer's style in-distribution but generates different reasoning at test time. STaR-DPO avoids this by training on the model's own self-generated rationales throughout. STaR-DPO matches the best explicit reward models in-distribution (73.9% vs 73-74%) while outperforming all of them OOD, particularly on Safety (91.0% vs 81.8% for PairRM). On reasoning-specific tasks (UltraInteract training, RewardBench Reasoning), STaR-DPO achieves 87.2% vs 70.8% for the best non-reasoning model.

3.3.3 Critique-out-Loud Reward Models (CLoud)

[Ankner et al., 2024] train reward models that generate a natural-language critique c before predicting a scalar reward. The model has 2 parameter groups built on top of a shared base θ_B: θ_{LM}: the pretrained LM head, which generates critique tokens autoregressively, and θ_R: a randomly initialised shallow MLP reward head, which reads the final hidden state of the full sequence and outputs a scalar reward. Scoring at inference is therefore a two-step procedure: (1) *generate* a critique by feeding (x, y) through $\theta_B + \theta_{LM}$ to autoregressively sample c; then (2) *score* by feeding the full sequence (x, y, c) through $\theta_B + \theta_R$ to obtain the scalar $r_{\theta_R}(x, y, c)$. The training pipeline is as follows:

Stage 1 — SFT on oracle critiques External oracle critiques are generated offline by Llama-3.1-405B for every chosen and rejected response. θ_B, θ_{LM} is fine-tuned on these via NLL; (θ_R does not yet exist):

$$\mathcal{L}_{\text{SFT}}(\theta_B, \theta_{LM}) = -\mathbb{E}\left[\sum_t \log p(c_t^- \mid x, y^-, c_{<t}^-) + \sum_t \log p(c_t^+ \mid x, y^+, c_{<t}^+)\right] \tag{22}$$

Stage 2 — Self-generated Critique Using the Stage 1 model, new critiques are sampled for every training example: $\hat{c}^{\pm} \sim p(\cdot \mid x, y^{\pm}; \tilde{\theta}_B, \tilde{\theta}_{LM})$. These replace the oracle critiques to form $\mathcal{D}_{\text{self}}$. At inference θ_R will see self-generated critiques, so we need to produce good self-generated critiques to reduce distributional shift.

Stage 3 — Joint training on $\mathcal{D}_{\text{self}}$. θ_R is randomly initialised on top of θ_B . All three parameter groups are jointly optimised on $\mathcal{D}_{\text{self}}$:

$$\mathcal{L}_{\text{CLoud}} = \underbrace{-\mathbb{E}\big[\log \sigma\big(r_{\theta_R}(x, y^+, \hat{c}^+) - r_{\theta_R}(x, y^-, \hat{c}^-)\big)\big]}_{\mathcal{L}_{\text{RM}}\text{: trains } \theta_R \text{ and } \theta_B} + \lambda \underbrace{\mathcal{L}_{\text{SFT}}(\theta_B, \theta_{LM}, \mathcal{D}_{\text{self}})}_{\text{trains } \theta_{LM} \text{ and } \theta_B} \tag{23}$$

$\mathcal{L}_{\text{RM}}$ sends gradients into θ_R via the classic Bradley-Terry preference modelling loss; $\mathcal{L}_{\text{SFT}}$ sends gradients into θ_{LM}. The $\mathcal{L}_{\text{SFT}}$ term in Stage 3 is important — without it, θ_{LM} receives no gradient and critique quality degrades, which then corrupts the conditioning signal seen by θ_R.

At inference, N critiques are sampled and the resulting scalar rewards averaged:

$$\bar{R}(x, y) = \frac{1}{N}\sum_{i=1}^{N} r_{\theta_R}(x, y, c_i)\,, \qquad c_i \sim p(\cdot \mid x, y;\, \theta_B, \theta_{LM}) \tag{24}$$

On-policy critiques are essential: training the reward head on oracle (off-policy) critiques causes a 5.6% drop in RewardBench accuracy for the 8B model (3.0 pp for 70B) relative to self-generated (on-policy) training. The reward head learns to condition on the statistical properties of the critique distribution it saw during training; oracle critiques are systematically longer and more detailed than self-generated ones, so the reward head cannot properly calibrate from a distribution shift critique at inference. CLoud-8B improves pairwise RewardBench accuracy by 4.65% over a classic RM and is Pareto-dominant across all $N \in [2, 16]$ on ArenaHard Best-of-N win rate. The 8B CLoud model outperforms the 70B classic RM on Chat and Safety categories.

3.3.4 Critic-RM: Critiques as Latent Variables

[Yu et al., 2025] provide a principled variational formulation that unifies critique generation and reward modeling, requiring no teacher LLM — critiques are self-generated and quality-filtered.

The model is built on a shared backbone M_θ with two heads: $q_\phi = h_g \circ M_\theta$ (the pretrained LM head, generates critique text) and $r_\psi = h_r \circ M_\theta$ (a randomly initialised scalar head, predicts reward). q_ϕ plays a dual role: it is simultaneously the *approximate posterior* over the latent critiques z (pushed toward the oracle p^* by ℓ_c) and the generator of critique samples that r_ψ conditions on in ℓ_r.

Introducing critiques for chosen and rejected responses z^+, z^- as latents and marginalising:

$$p(y^+ \succ y^- \mid x) = \sum_{z^+, z^-} p(y^+ \succ y^- \mid z^+, z^-, x)\, p^*(z^+ \mid y^+, x)\, p^*(z^- \mid y^-, x)$$

Since p^* is intractable, substitute the learned q_ϕ and apply Jensen's inequality to obtain the ELBO:

$$\log p(y^+ \succ y^- \mid x) \;\geq\; \mathbb{E}_{q_\phi(z^+|y^+,x),\, q_\phi(z^-|y^-,x)}\left[\log \frac{p(y^+ \succ y^-,\, z^+, z^- \mid x)}{q_\phi(z^+|y^+, x)\, q_\phi(z^-|y^-, x)}\right] \tag{25}$$

Maximising this ELBO decomposes into two objectives. The **critique generation loss** ℓ_c trains q_ϕ to approximate p^*. Reverse KL would cause entropy collapse, so *forward* KL is used, treating the filtered critique set Z as a uniform proxy for p^*:

$$\ell_c = -\frac{1}{|Z|}\sum_{z \in Z} \log q_\phi(z \mid y, x) \tag{26}$$

The **preference modeling loss** ℓ_r trains r_ψ: the critique z is appended to the response, and the scalar head pools from the concatenation $[y; z]$:

$$\ell_r = -\log \sigma\big(r_\psi(x, [y^+; z^+]) - r_\psi(x, [y^-; z^-])\big) \tag{27}$$

Dynamic weighting schedule. The two losses are combined with a time-varying weight $\lambda(t)$:

$$\mathcal{L}(\phi, \psi) = \lambda(t) \cdot \ell_c + \big(1 - \lambda(t)\big) \cdot \ell_r, \qquad \lambda(t) = \begin{cases} 1 & t < (K-1)T \\ 1 - \beta \cdot \dfrac{t - (K-1)T}{T} & t \geq (K-1)T \end{cases} \tag{28}$$

With $K = 2$ epochs and $\beta = 0.9$: the first epoch trains *only* on critique generation ($\lambda = 1$, reward head receives no gradient); in the final epoch λ decays from 1 to 0.1, gradually shifting supervision to reward prediction. This ordering is essential — the reward head only begins training once the LM already has strong critique capability, preventing reward-head overfitting before useful critiques can be generated. At inference, m critiques are sampled and scalar rewards averaged: $\bar{r} = \frac{1}{m}\sum_{i=1}^{m} r_\psi(x, [y, z_i])$.

3.4 Training the Base Model with Critique

The methods in Section 3.3 improve reward models and judges by having them generate critique. A closely related cluster instead trains the base *policy* model directly using critique signals. This section covers the methods most directly related to Method 1.

3.4.1 Critique Fine-Tuning

[Y. Wang et al., 2025] ask: what happens if we use *only* the NLL term on critique data, without any preference optimisation? In Critique Fine-Tuning (CFT), the base model is trained on the teacher's critique c_v of a given query-response pair by maximising:

$$\mathcal{L}_{\text{CFT}}(\theta) = -\sum_{t=1}^{|c_v|} \log \pi_\theta(c_{v,t} \mid c_{v,<t}, [x; y]) \tag{29}$$

where $[x; y]$ concatenates the query and a (potentially flawed) response, and c_v is a critique generated by a teacher model (GPT-4o). At inference, the CFT-trained model generates direct answers without producing critiques.

3.4.2 SuperCorrect: Cross-Model Collaborative DPO

[Yang et al., 2024] introduce a two-stage framework. Stage 1 distils hierarchical thought templates from a large teacher model (o1-preview) via SFT. Stage 2 applies **cross-model collaborative DPO** on teacher-guided error-correction traces. Given an erroneous student solution with the first error at step k, the teacher generates a correction trace (a_k^+, c_k^+) (error analysis and correction guidance) while the student independently generates (a_k^-, c_k^-). The DPO loss is:

$$\mathcal{L}_{\text{Cross-DPO}} = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(a_k^+, c_k^+ \mid x, s_{<k})}{\pi_{\text{ref}}(a_k^+, c_k^+ \mid x, s_{<k})} - \beta \log \frac{\pi_\theta(a_k^-, c_k^- \mid x, s_{<k})}{\pi_{\text{ref}}(a_k^-, c_k^- \mid x, s_{<k})}\right)\right] \tag{30}$$

where $s_{<k}$ are the correct preceding steps. The teacher's correction trace plays the role of c_v in Method 1. SuperCorrect-7B achieves 70.2% on MATH (surpassing DeepSeekMath-7B by 7.8%) and demonstrates that cross-model DPO at the step level substantially improves self-correction capabilities. CriticGPT [McAleese et al., 2024] and CTRL [Xie et al., 2025] provide complementary evidence that LLMs can be explicitly taught to generate high-quality critiques.

References

Rafailov, Rafael et al. (2024). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. arXiv: 2305.18290 [cs.LG]. URL: https://arxiv.org/abs/2305.18290.

Wolfe, Cameron R. (2025). *Direct Preference Optimization (DPO)*. Blog post. URL: https://cameronrwolfe.substack.com/p/direct-preference-optimization.

Romero, Tyler (2024). *Direct Preference Optimization Explained In-depth*. Blog post. URL: https://www.tylerromero.com/posts/2024-04-dpo/.

Bai, Yuntao et al. (2022). *Constitutional AI: Harmlessness from AI Feedback*. arXiv: 2212.08073 [cs.CL]. URL: https://arxiv.org/abs/2212.08073.

Lee, Harrison et al. (2024). *RLAIF vs. RLHF: Scaling Reinforcement Learning from Human Feedback with AI Feedback*. arXiv: 2309.00267 [cs.CL]. URL: https://arxiv.org/abs/2309.00267.

Li, Ang et al. (2024). *HRLAIF: Improvements in Helpfulness and Harmlessness in Open-domain Reinforcement Learning From AI Feedback*. arXiv: 2403.08309 [cs.LG]. URL: https://arxiv.org/abs/2403.08309.

Liu, Hanbing et al. (2025). *Uncovering the Impact of Chain-of-Thought Reasoning for Direct Preference Optimization*. arXiv: 2502.11656 [cs.CL]. URL: https://arxiv.org/abs/2502.11656.

Zelikman, Eric et al. (2022). *STaR: Bootstrapping Reasoning With Reasoning*. arXiv: 2203.14465 [cs.LG]. URL: https://arxiv.org/abs/2203.14465.

Hosseini, Arian et al. (2024). *V-STaR: Training Verifiers for Self-Taught Reasoners*. arXiv: 2402.06457 [cs.LG]. URL: https://arxiv.org/abs/2402.06457.

Pang, Richard Yuanzhe et al. (2024). *Iterative Reasoning Preference Optimization*. arXiv: 2404.19733 [cs.CL]. URL: https://arxiv.org/abs/2404.19733.

Shen, Yaojie et al. (2024). *AIPO: Improving Training Objective for Iterative Preference Optimization*. arXiv: 2409.08845 [cs.CL]. URL: https://arxiv.org/abs/2409.08845.

Wang, Peifeng et al. (2024). *Direct Judgement Preference Optimization*. arXiv: 2409.14664 [cs.CL]. URL: https://arxiv.org/abs/2409.14664.

Zhang, Lunjun et al. (2024). *Generative Verifiers: Reward Modeling as Next-Token Prediction*. arXiv: 2408.15240 [cs.LG]. URL: https://arxiv.org/abs/2408.15240.

Mahan, Dakota et al. (2024). *Generative Reward Models*. arXiv: 2410.12832 [cs.LG]. URL: https://arxiv.org/abs/2410.12832.

Ankner, Zachary et al. (2024). *Critique-out-Loud Reward Models*. arXiv: 2408.11791 [cs.LG]. URL: https://arxiv.org/abs/2408.11791.

Yu, Yue et al. (2025). *Self-Generated Critiques Boost Reward Modeling for Language Models*. arXiv: 2411.16646 [cs.CL]. URL: https://arxiv.org/abs/2411.16646.

Wang, Yubo, Xiang Yue, and Wenhu Chen (2025). *Critique Fine-Tuning: Learning to Critique is More Effective than Learning to Imitate*. arXiv: 2501.17703 [cs.CL]. URL: https://arxiv.org/abs/2501.17703.

Yang, Ling et al. (2024). *SuperCorrect: Advancing Small LLM Reasoning with Thought Template Distillation and Self-Correction*. arXiv: 2410.09008 [cs.CL]. URL: https://arxiv.org/abs/2410.09008.

McAleese, Nat et al. (2024). *LLM Critics Help Catch LLM Bugs*. arXiv: 2407.00215 [cs.CL]. URL: https://arxiv.org/abs/2407.00215.

Xie, Zhihui et al. (2025). *Teaching Language Models to Critique via Reinforcement Learning*. arXiv: 2502.03492 [cs.CL]. URL: https://arxiv.org/abs/2502.03492.